UADE

UADE

Programación III – Diseño y Análisis de Algoritmos

Temas a desarrollar

1. Repaso clase anterior
2. Ejercicio de la Práctica
3. Poda por cota inferior y/o superior
4. Cota aplicada a asignación de Tareas
5. Problema de la mochila con cota

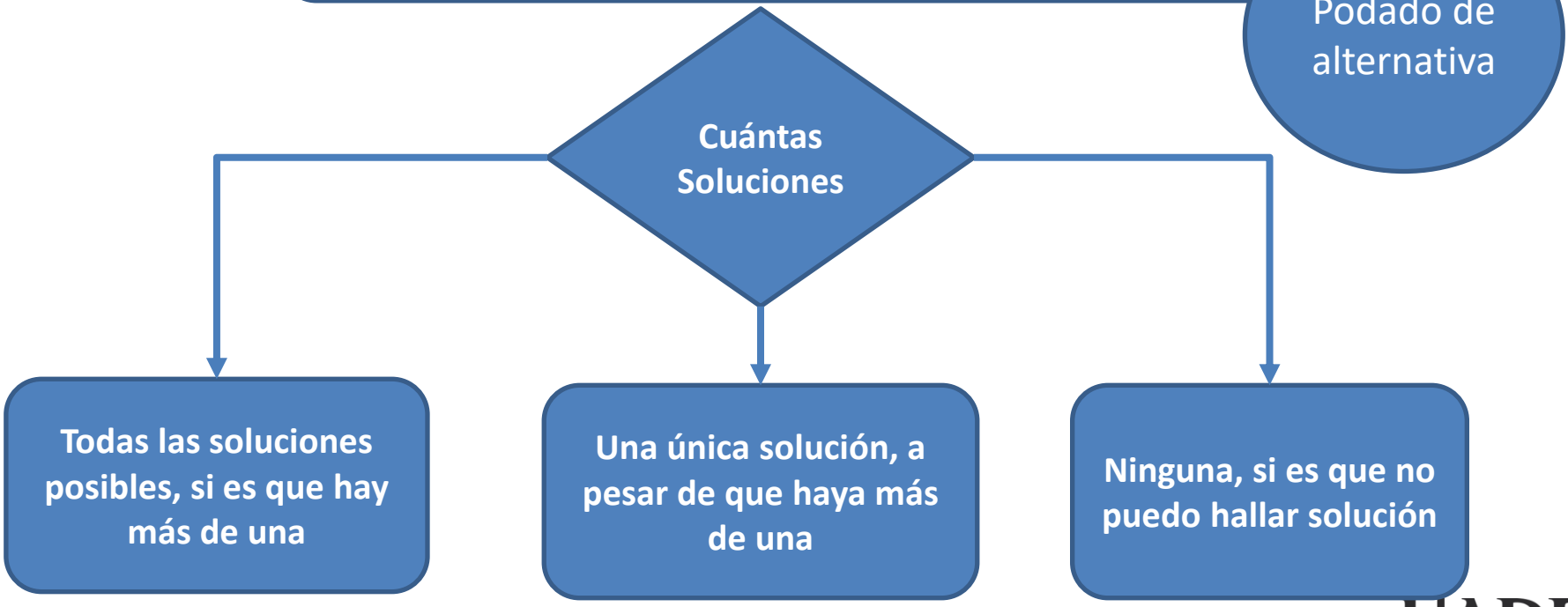
Repaso de Backtracking

Backtracking

La solución posible se basa es la exploración directa de todas las posibilidades

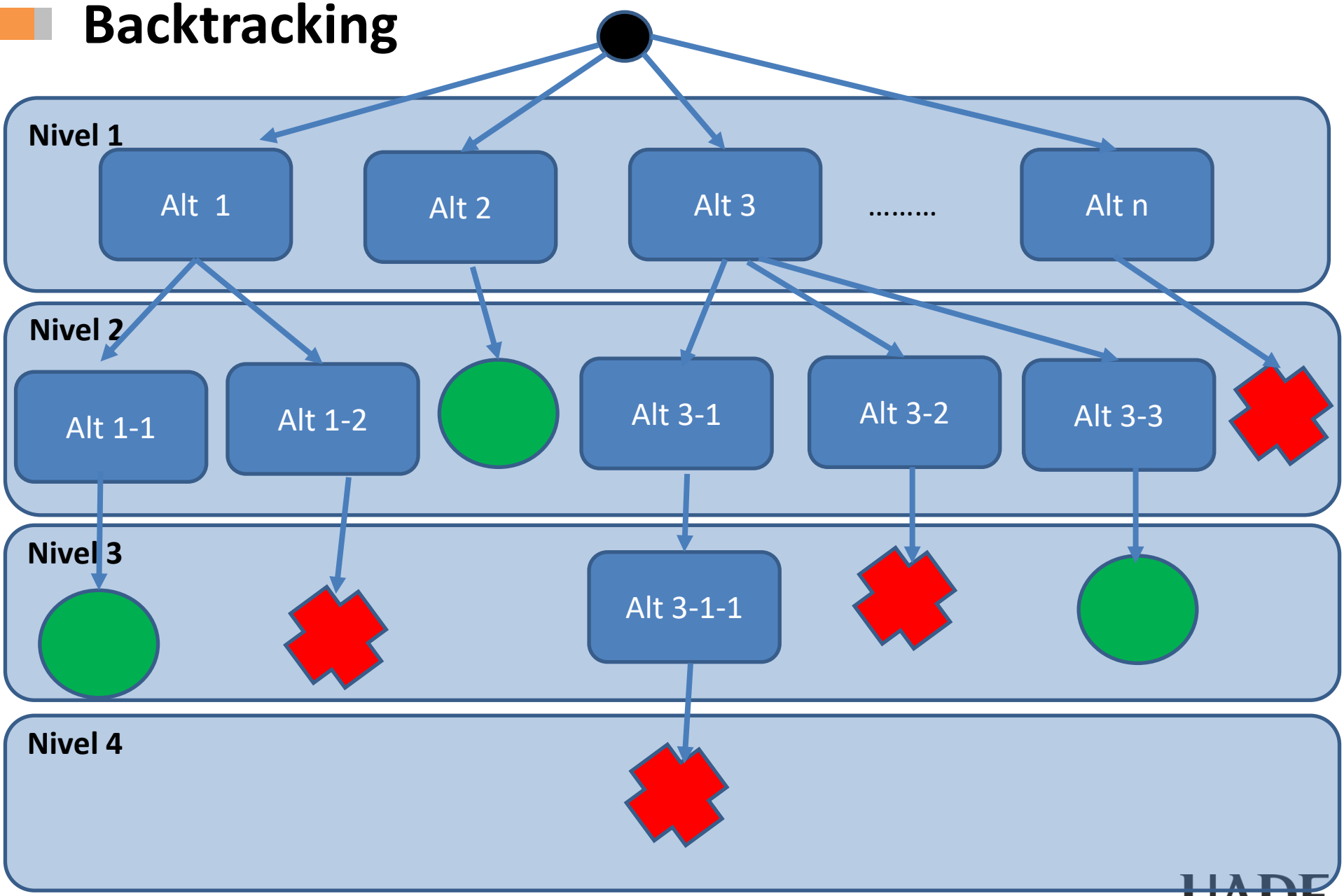
Es un método de búsqueda de soluciones exhaustivas sobre grafos dirigidos acíclicos

Podado de alternativa



UADE

Backtracking



Backtracking – Esquema General

ALGORITMO BACKTRACKING

Entrada: S: Solución parcial, E: etapa

```
Valores ← generarValoresPosibles(S, E)
para cada v ∈ Valores
   agregar(S, E, v)
   si factible(S)
      si esSolucion(S)
         mostrar(S)
      sino
         backtracking(S, E + 1)
      fin si
   fin si
   eliminar(S, E)
fin para
```

Resolución problema práctica

UADE

Ejercicio

- Plantear la estrategia por backtracking
- Desarrollar pseudocódigo
- Calcular complejidad temporal teórica

Ejercicio 1. Se tienen ***m*** procesadores idénticos y ***n*** tareas con un tiempo de ejecución dado. Se requiere encontrar una asignación de tareas a procesadores de manera de minimizar el tiempo de ejecución del total de tareas.

¿Qué es una cota?

¿Qué es una cota?

- En backtracking, las **cotas** (bounds su termino en ingles) se uilizan para decidir si conviene seguir explorando una rama del árbol de búsqueda o podarla (descartarla). Son estimaciones del valor que puede alcanzar una solución parcial que se esta construyendo.

- En general se van a utilizar dos tipos de cota, inferior y superior.

Backtracking

Cota Inferior

Es un límite mínimo del valor que puede alcanzar una solución que se extiende desde una solución parcial:

- Representa el mejor caso posible que podrías lograr continuando por esa rama.
- En problemas de minimización, indica el menor costo posible que podría llegar a tener esa rama.
- Si esa cota inferior ya es peor que la mejor solución encontrada hasta el momento, se poda la rama porque no vale la pena seguir.

Backtracking

Cota Superior

Es un límite máximo del valor que puede alcanzar una solución en esa rama:

- ✓ Representa una estimación optimista del mejor valor posible a alcanzar (aunque no sea el real).
- ✓ En problemas de maximización, indica el máximo beneficio posible que podría lograrse desde esa rama.
- ✓ Si esa cota superior es menor que la mejor solución ya encontrada, también se puede podar.

Ejercicio con cálculo de cota inicial

- Diseñar a través de un problema Greedy la obtención de una cota inicial para obtener como poda

Ejercicio 1. Se tienen ***m*** procesadores idénticos y ***n*** tareas con un tiempo de ejecución dado. Se requiere encontrar una asignación de tareas a procesadores de manera de minimizar el tiempo de ejecución del total de tareas.

Cota Inicial Problema Asignación Tareas

- Un algoritmo Greedy que puede obtener una solución "suficientemente buena" utiliza el siguiente criterio:
  - Selección: tarea de mayor tiempo de ejecución
  - Criterio adicional: siempre asignar esa tarea al procesador más libre.

Cota Inicial Problema Asignación Tareas

- Un algoritmo Greedy que puede obtener una solución "suficientemente buena" utiliza el siguiente criterio:
  - Selección: tarea de mayor tiempo de ejecución
  - Criterio adicional: siempre asignar esa tarea al procesador más libre.

Cota Inicial Problema Asignación Tareas

Supongamos un ejemplo de dos procesadores y las tareas {8, 7, 6, 5, 4}, si se aplica el algoritmo antes mencionado el resultado sería el siguiente:

Procesador 1: {8,5,4}

Procesador 2: {7,6}

El tiempo máximo lo tendría el procesador 1 con 17, aunque la asignación óptima tendría 15:

Procesador 1: {8,7}

Procesador 2: {6,5,4}

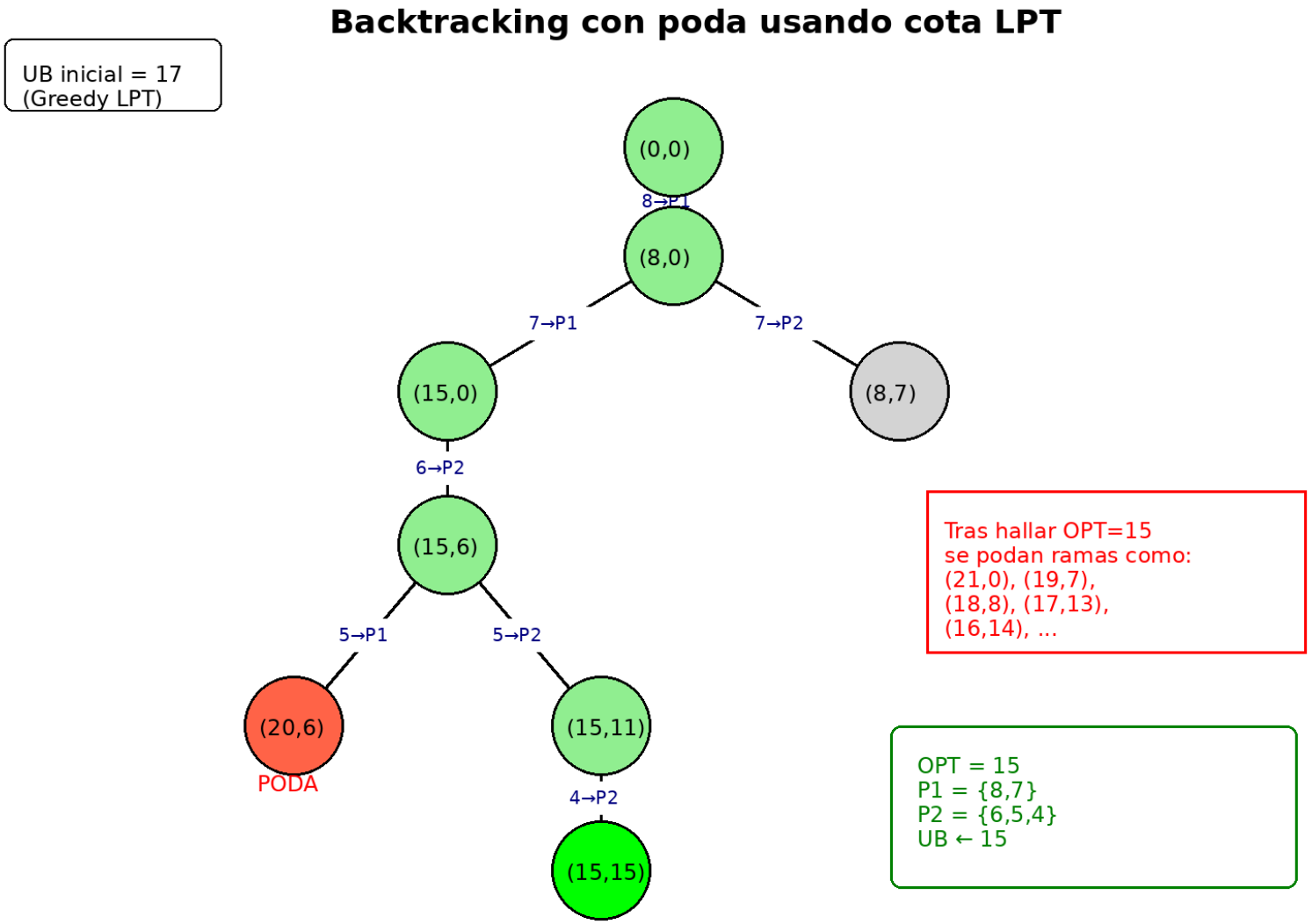
Cota Inicial Asignación Tareas - Pseudocódigo

```
Algoritmo AsignarTareasGreedy
Entradas: Vector<entero>  tareas, Entero  m
Salida: Vector<entero> asignación
  Vector<real> carga ← InicializarVector()
  Vector<entero> asignacion ← InicializarVector()
   Entero i, j, p
   Entero mejorProcesador
  MergeSort(tareas)  // ordenar tareas en forma decreciente por tiempo de ejecución
  Para j ← 1 hasta m
              carga[j] ← 0
  Fin Para
  Para i ← 1 hasta longitud(tareas)
          mejorProcesador ← 1
                      Para p ← 2 hasta m
                         Si carga[p] < carga[mejorProcesador]
                           mejorProcesador ← p
                         Fin Si
                      Fin Para
         asignacion[i] ← mejorProcesador
         carga[mejorProcesador] ← carga[mejorProcesador] + tareas[i]
   Fin Para
  devolver asignacion
fin AsignarTareasGreedy
```

Asignación Tareas – Complejidad Temporal

```
Algoritmo AsignarTareasGreedy
Entradas: Vector<entero>  tareas, Entero  m
Salida: Vector<entero> asignación
  Vector<real> carga ← InicializarVector()
  Vector<entero> asignacion ← InicializarVector()
   Entero i, j, p
   Entero mejorProcesador
  MergeSort(tareas)  // ordenar tareas en forma decreciente por tiempo de ejecución   O(t log t)
  Para j ← 1 hasta m                                                                  O(m)
            carga[j] ← 0
  Fin Para
  Para i ← 1 hasta longitud(tareas)                                                   O(t)
        mejorProcesador ← 1
                  Para p ← 2 hasta m                                                  O(m)
                    Si carga[p] < carga[mejorProcesador]
                      mejorProcesador ← p
                    Fin Si
                  Fin Para                                                            O(t log t + t*m)
        asignacion[i] ← mejorProcesador
        carga[mejorProcesador] ← carga[mejorProcesador] + tareas[i]
   Fin Para
  devolver asignacion
fin AsignarTareasGreedy
```

Asignación Tareas – Backtracking usando Cota



Problema de la mochila

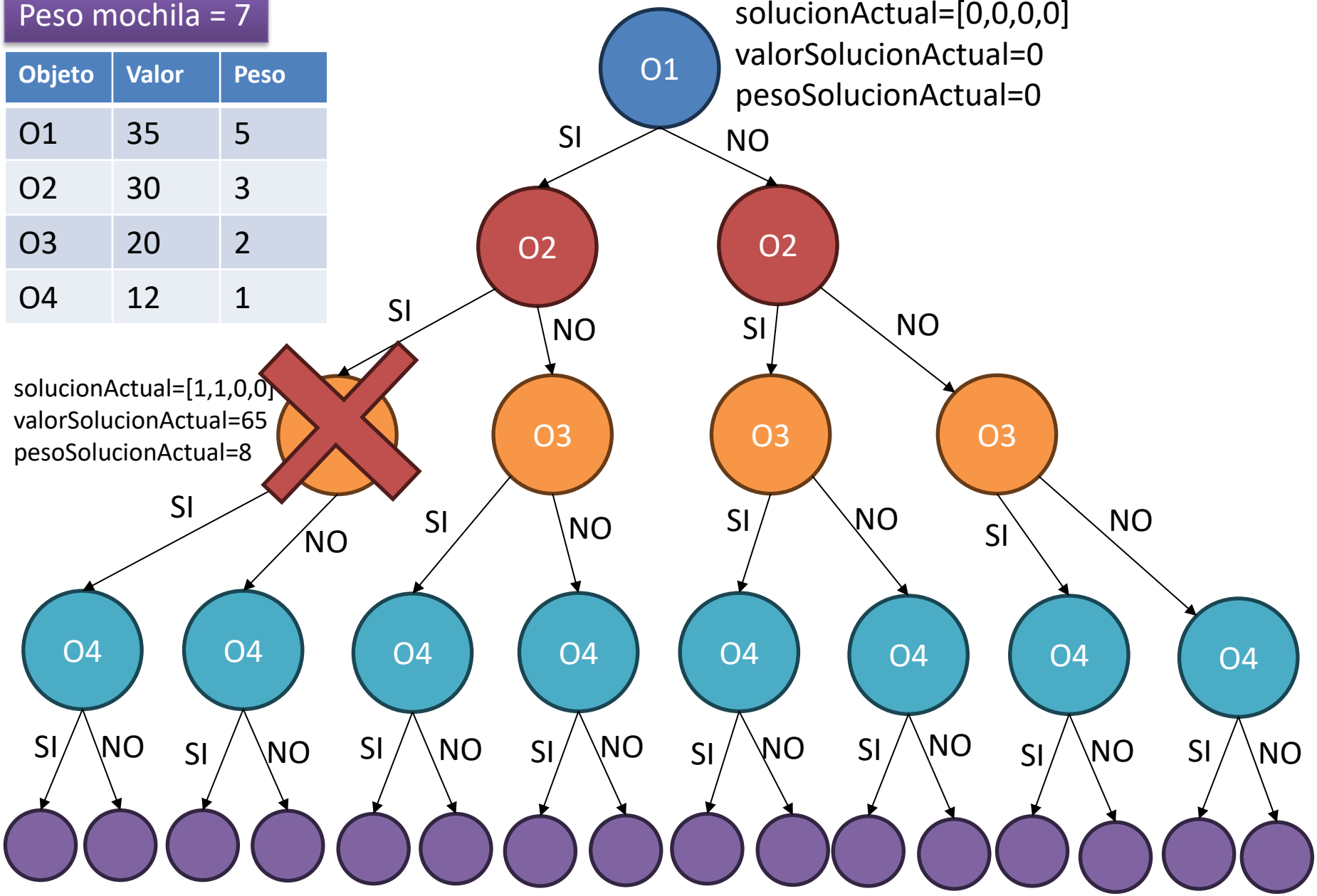
Problema de la Mochila Greedy

El problema consiste en que se tienen **n** objetos y una mochila. Para ***i = 1; 2; ..n***, el objeto ***i*** tiene un peso positivo ***Pi*** y un valor positivo ***Vi***. La mochila puede llevar un peso que no sobrepase **P**. El objetivo es llenar la mochila de tal manera que se maximice el valor de los objetos transportados, respetando la limitación de capacidad impuesta.

Existen varias versiones del problema. En una de ellas, los objetos se pueden fraccionar: si una fracción ***Xi*** (0 <= Xi <= 1) del objeto ***i*** es ubicada en la mochila contribuye en ***Xi * Pi*** al peso total de la mochila y en ***Xi * Vi*** al valor de la carga. En otras versiones no se puede fraccionar y se lo llama "Mochila 0-1".

Peso mochila = 7

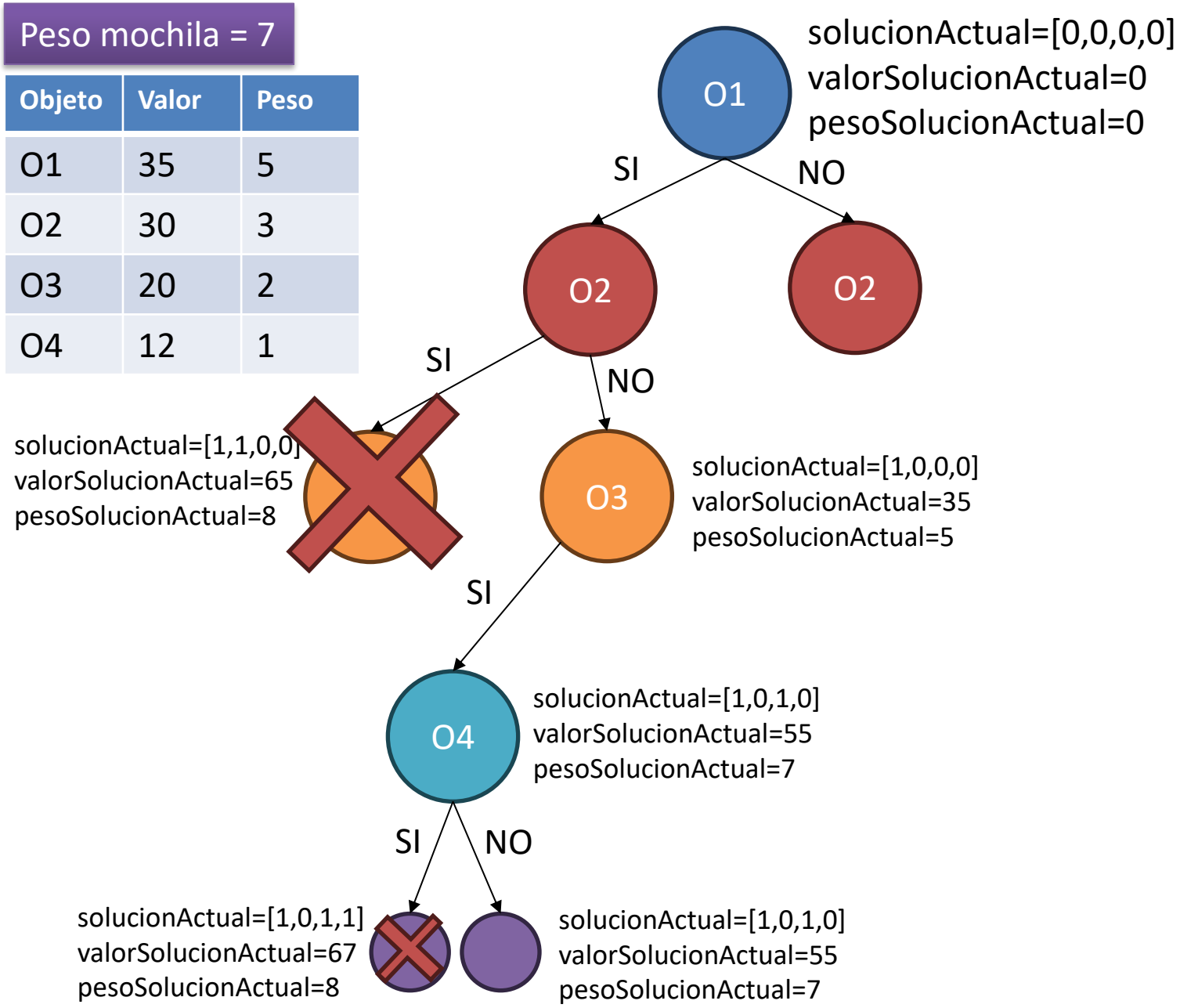
Objeto	Valor	Peso
O1	35	5
O2	30	3
O3	20	2
O4	12	1



UADE

Peso mochila = 7

Objeto	Valor	Peso
O1	35	5
O2	30	3
O3	20	2
O4	12	1



UADE

Pseudocódigo

```
Algoritmo Mochila
Entrada: Objetos: Vector<Objeto>, capacidad: entero, mejorSolucion: Vector<entero>, valorMejorSolucion: entero, solucionActual
Vector<entero>,  valorSolucionActual: entero, pesoSolucionActual: entero,etapa:entero
Salida: entero

para i = 0 hasta 1:
        solucionActual[etapa] <- i
        pesoSolucionActual <- pesoSolucionActual + i * Objetos[etapa].peso
        valorSolucionActual <- valorSolucionActual + i * Objetos[etapa].valor
        si pesoSolucionActual == capacidad:
                si valorMejorSolucion = -1 O valorMejorSolucion < valorSolucionActual
                                mejorSolucion <- solucionActual
                                valorMejorSolucion <-  valorSolucionActual
                fin si
        fin si
        si pesoSolucionActual < capacidad:
                si etapa = longitud(Objetos) - 1
                        si valorMejorSolucion = -1 O valorMejorSolucion < valorSolucionActual
                                mejorSolucion <- solucionActual
                                valorMejorSolucion <-  valorSolucionActual
                        fin si
                fin si
                sino:
                        valorMejorSolucion  <- Mochila(Objetos, capacidad, mejorSolucion, valorMejorSolucion,
                                                        solucionActual, valorSolucionActual, etapa + 1)
                fin sino
         fin si
fin para
devolver valorMejorSolucion
```

Complejidad Temporal

Algoritmo Mochila
Entrada: Objetos: Vector<Objeto>, capacidad: entero, mejorSolucion: Vector<entero>, valorMejorSolucion: entero, solucionActual Vector<entero>, valorSolucionActual: entero, pesoSolucionActual: entero,etapa:entero
Salida: entero

```
para i = 0 hasta 1:
        solucionActual[etapa] <- i
        pesoSolucionActual <- pesoSolucionActual + i * Objetos[etapa].peso
        valorSolucionActual <- valorSolucionActual + i * Objetos[etapa].valor
        si pesoSolucionActual == capacidad:
                si valorMejorSolucion = -1 O valorMejorSolucion < valorSolucionActual
                        mejorSolucion <- solucionActual
                        valorMejorSolucion <-  valorSolucionActual
                fin si
        fin si
        si pesoSolucionActual < capacidad:
                si etapa = longitud(Objetos) - 1
                        si valorMejorSolucion = -1 O valorMejorSolucion < valorSolucionActual
                                mejorSolucion <- solucionActual
                                valorMejorSolucion <-  valorSolucionActual
                        fin si
                fin si
                sino:
                        valorMejorSolucion  <- Mochila(Objetos, capacidad, mejorSolucion, valorMejorSolucion,
                                                        solucionActual, valorSolucionActual, etapa + 1)
                fin sino
         fin si
fin para
devolver valorMejorSolucion
```

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < 1 \\ \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(a^{n \ div \ b}) & \text{si } a > 1 \end{cases}$$

a=2; b=1; k=0

$$\mathcal{O}(2^n)$$

UADE

Poda

En el momento de expandir y agregar nuevos nodos, se puede determinar que algunos de ellos no conducirán a una solución y se “poda” esa rama, es decir, se deja de analizar.

Se puede definir una *cota*: valor de la mejor solución que se podría alcanzar desde ese nodo

- Si el mejor valor que se podría alcanzar desde ese nodo es peor que el valor de la mejor solución encontrada hasta el momento, se puede podar el nodo.

Poda extra utilizando una cota superior

Se van a utilizar dos tipos de poda para el problema de la mochila:

- **Poda por factibilidad:** si el peso actual excede la capacidad no sigo por esa rama (ya es inválida, nunca será solución).

- **Poda por cota superior** (o por optimalidad): si incluso en el mejor de los casos no puedo superar la mejor solución encontrada, no tiene sentido seguir explorando esa rama.

Poda extra utilizando una cota superior

- La **cota superior** es una **estimación optimista del mejor valor** que podríamos lograr siguiendo por esa rama.

- Para calcularla, usamos la idea del **problema de la mochila fraccional (Greedy)**. La mayoría de los casos no serán soluciones válidas para el problema 0-1 pero sirve como **estimación máxima posible**.

- Si aunque se tomen fracciones no se puede superar la mejor solución actual, ya no vale la pena seguir.

Pseudocódigo con poda por cota superior

```
Algoritmo Mochila
Entrada: Objetos: Vector<Objeto>, capacidad: entero, mejorSolucion: Vector<entero>, valorMejorSolucion: entero,
solucionActual: Vector<entero>,  valorSolucionActual: entero, pesoSolucionActual: entero,etapa:entero
Salida: entero

para i = 0 hasta 1:
        solucionActual[etapa] <- i
        pesoSolucionActual <- pesoSolucionActual + i * Objetos[etapa].peso
        valorSolucionActual <- valorSolucionActual + i * Objetos[etapa].valor
        si pesoSolucionActual == capacidad:
                si valorMejorSolucion = -1 O valorMejorSolucion < valorSolucionActual
                        mejorSolucion <- solucionActual
                        valorMejorSolucion <-  valorSolucionActual
                fin si
        fin si
        si pesoSolucionActual < capacidad:
                cotaSuperior = calcularCotaSuperior(Objetos, solucionActual, etapa)
                si cotaSuperior > valorMejorSolucion:
                        si etapa = longitud(Objetos) - 1
                             si valorMejorSolucion = -1 O valorMejorSolucion < valorSolucionActual
                                    mejorSolucion <- solucionActual
                                    valorMejorSolucion <-  valorSolucionActual
                             fin si
                        fin si
                        sino:
                             valorMejorSolucion  <- Mochila(Objetos, capacidad, mejorSolucion, valorMejorSolucion,
                                                            solucionActual, valorSolucionActual, etapa + 1)
                        fin sino
                fin si
         fin si
fin para
devolver valorMejorSolucion
```

¡Muchas gracias!

UADE